ILGC Tracker

AI-Powered Campus Housekeeping & Hygiene Compliance Tracker

Complete System Documentation — A to Z

Problem Statement: There have been recurring housekeeping issues on campus, particularly the failure to refill essential hygiene supplies such as tissues and soaps for continuous periods of up to four days. This lapse highlights the need for more consistent monitoring and timely replenishment of basic sanitation resources to maintain acceptable hygiene standards across campus.

Node.js | Python | PostgreSQL | Redis | Docker | Machine Learning | React Native

59	11,500+	50+	6
Source Files	Lines of Code	API Endpoints	Docker Services

Version 3.1.0 • Generated: 14 March 2026

Project: /Users/maruteymani/Documents/ILGC Tracker

Table of Contents

1. Project Overview
2. Technologies Used
3. System Architecture Diagram
4. Docker Container Map
5. Database Schema (ER Diagram)
6. Request Flow Sequences
7. AI/ML Pipeline Diagram
8. Every File Explained
    a. Root Level Files
    b. Backend — Node.js API
    c. AI Service — Python
    d. Mobile App — React Native
    e. Infrastructure — Nginx
9. Complete API Endpoints (50+)
10. User Roles & Permissions
11. ML Models Explained
12. Security Features
13. Deployment Guide
14. File Count Summary
15. Changelog

1. Project Overview

ILGC Tracker is an **AI-powered Campus Housekeeping & Hygiene Compliance Tracker** designed to solve a persistent problem on campus: the failure to refill essential hygiene supplies (tissues, soaps, sanitizers) for days at a time, and the lack of consistent monitoring of housekeeping standards. The system gives residents an instant way to report shortages and hygiene issues, uses machine learning to auto-categorize and prioritize those complaints, and gives administrators real-time visibility into compliance gaps — ensuring timely replenishment and accountability.

The Problem: Recurring housekeeping lapses on campus — essential hygiene supplies (tissues, soaps) go unrefilled for up to **4 consecutive days**. There is no structured system for residents to report shortages, no tracking of replenishment schedules, and no accountability mechanism for housekeeping staff. This lapse in basic sanitation resources poses health and hygiene risks campus-wide.

The Solution — ILGC Tracker: Track-based login (Admin / Housekeeping / Faculty-Student) with role-specific access (Admin/Supervisor/Cleaning Staff/Student&Faculty) • Building/Floor/Direction-wise washroom data tracking • Supply inventory monitoring (soap, tissue, sanitizer levels) • AI complaint analysis with 12 categories (including hygiene, supplies, cleaning) • Priority prediction (hygiene issues auto-flagged high/urgent) • **2-way work verification** (LLM image analysis + supervisor approval) • False reporting detection • Supervisor consumption-based reminders • Restocking timeline tracking • NLP sentiment analysis • AI auto-categorization with review flow • Complaint detail popup with full AI breakdown • Image upload for photographic evidence • Predictive maintenance to anticipate supply shortages • Light/Dark theme • Real-time dashboard for monitoring compliance • Mobile app shell for on-the-go access

How It Works (Simple)

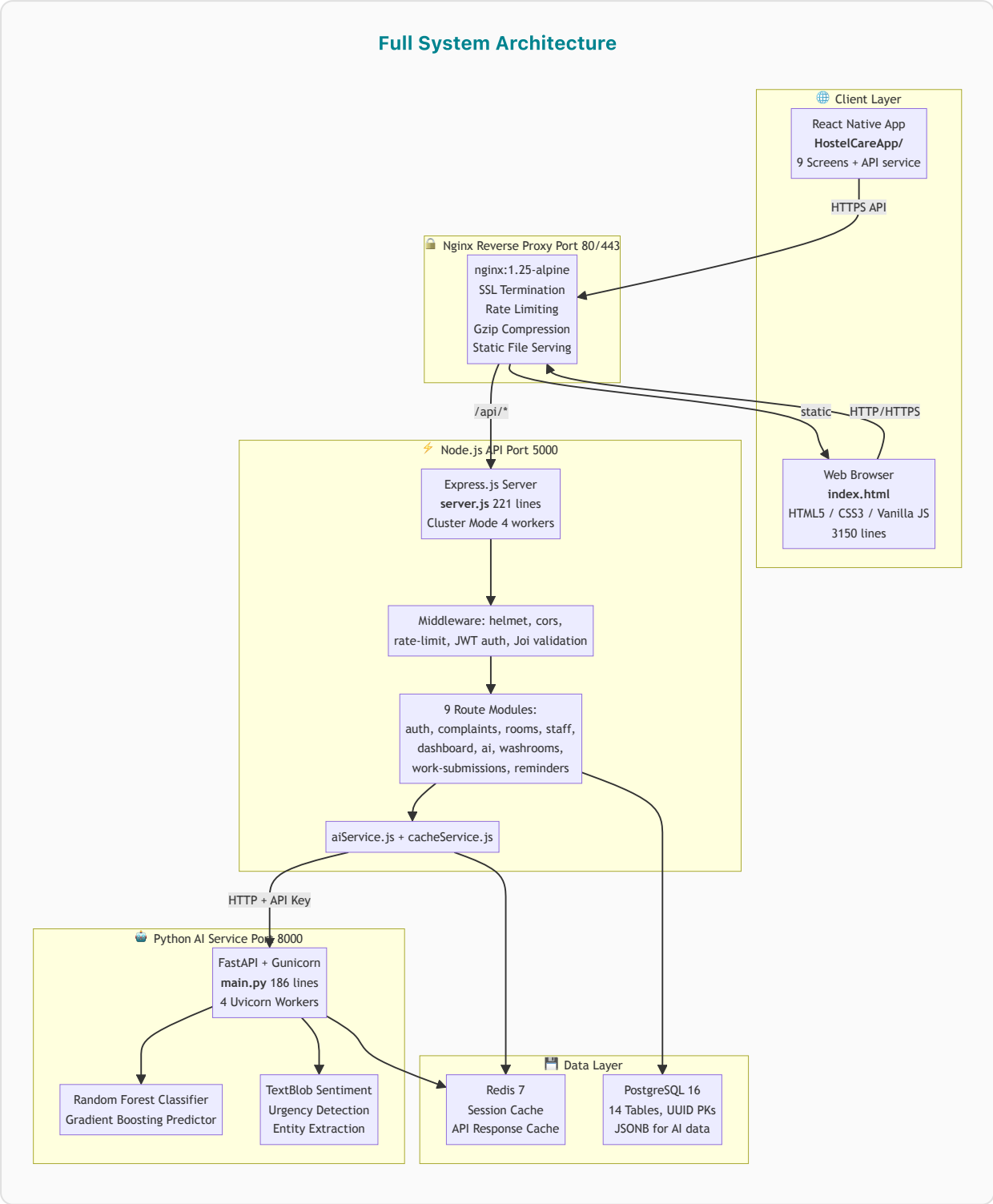
1. **Student/Faculty** notices that soap dispensers in the H2 – 2nd Floor South Wing washroom have been empty for 2 days. They open ILGC Tracker and submit a complaint: *"No soap in H2 2nd floor south wing washroom for 2 days, tissue dispensers also empty"* — attaches a photo as evidence, lets AI auto-detect the category
2. **AI automatically** categorizes it as "hygiene" (with confidence scores across all 12 categories), predicts "high" priority (hygiene shortages get elevated priority), analyzes negative sentiment, detects urgency keywords ("empty", "2 days"), extracts entities ("soap", "tissue", "H2 2nd floor south wing"), and suggests actions ("Restock soap and tissue dispensers immediately")
3. **Student reviews** AI-detected category + scores in a picker, confirms or changes, then submits
4. **Admin/Supervisor** sees the complaint on the dashboard with AI tags (category, sentiment, urgency, priority). Clicks the card → **detail popup** shows resident info, room/location, attached photo evidence, full AI analysis with confidence bars, suggested actions, estimated resolution time. Admin can escalate false reports to supervisors; supervisors assign complaints to their own staff
5. **Cleaning Staff** receives the task, restocks supplies, uploads proof photo. **LLM verifies** the uploaded image for cleanliness/completion. **Supervisor reviews** and approves the submission — *2-way verification ensures accountability*
6. **System** tracks supply levels building-wise (soap, tissue, sanitizer), sends consumption-based reminders to supervisors, detects false reporting patterns, logs everything with timestamps — admin can identify patterns (e.g., which buildings/floors have repeated shortages, restocking timelines) to improve scheduling

2. Technologies Used

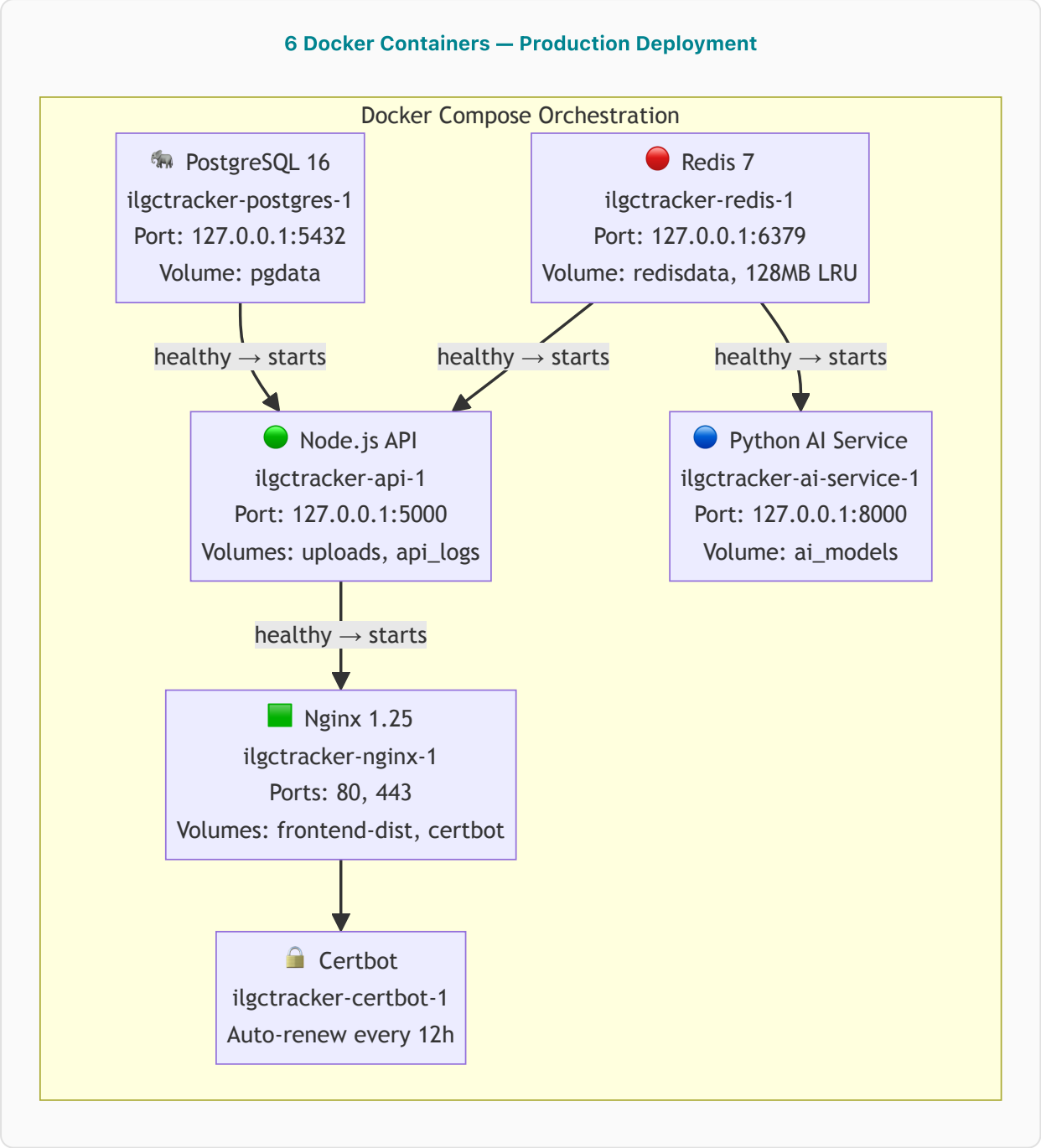
Layer	Technology	Language	Purpose

Frontend	Vanilla HTML/CSS/JS	HTML5 JS	Single-page web application (3,884 lines) — role-track login, admin/supervisor workflow controls, complaint detail modals, washroom/supply dashboards
Mobile	React Native 0.73 + WebView	JavaScript	Android + iOS shell that loads website UI with cache-busting, deep-link handling, and fallback host support
Backend API	Express.js 4.18	Node.js 18+	REST API server (9 route modules)
AI Service	FastAPI + Gunicorn	Python 3.11	Machine learning microservice
Database	PostgreSQL 16	SQL	Primary data store (14 tables)
Cache	Redis 7	—	Caching + session store
ML Models	scikit-learn	Python	Random Forest + Gradient Boosting
NLP	TextBlob + NLTK + spaCy	Python	Sentiment, urgency, entity extraction
Reverse Proxy	Nginx 1.25	—	SSL termination, rate limiting, static files
Auth	JWT (jsonwebtoken)	—	Token-based authentication (access + refresh)
ORM	Knex.js	JS	SQL query builder + migrations
Containers	Docker + Compose	Docker	Production deployment (6 services)
SSL	Certbot / Let's Encrypt	—	HTTPS certificates (auto-renew)
Logging	Winston + Structlog	—	Structured logging with rotation
Monitoring	Prometheus	—	Request/ML prediction metrics

3. System Architecture



4. Docker Container Map



Container	Image	Port	Volumes	Health Check
PostgreSQL	postgres:16-alpine	127.0.0.1:5432	pgdata	pg_isready
Redis	redis:7-alpine	127.0.0.1:6379	redisdata	redis-cli ping
Node API	ilgctracker-api (custom)	127.0.0.1:5000	uploads, api_logs	wget /health
AI Service	ilgctracker-ai-service (custom)	127.0.0.1:8000	ai_models	python httpx /health
Nginx	nginx:1.25-alpine	80, 443	frontend-dist, certbot	wget /health
Certbot	certbot/certbot	—	certbot_conf, certbot_www	—

5. Database Schema (14 Tables)

Entity Relationship Diagram

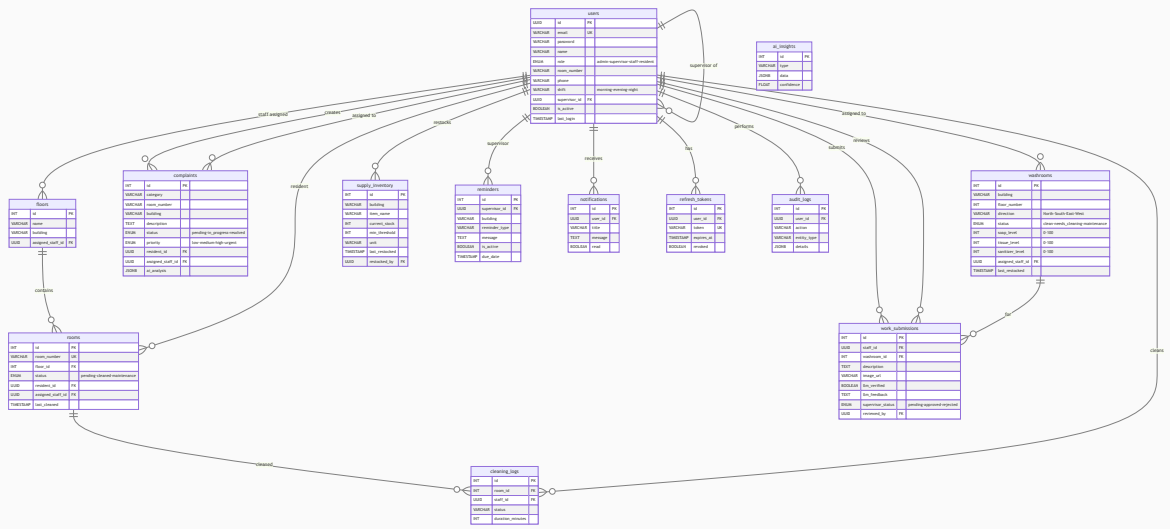
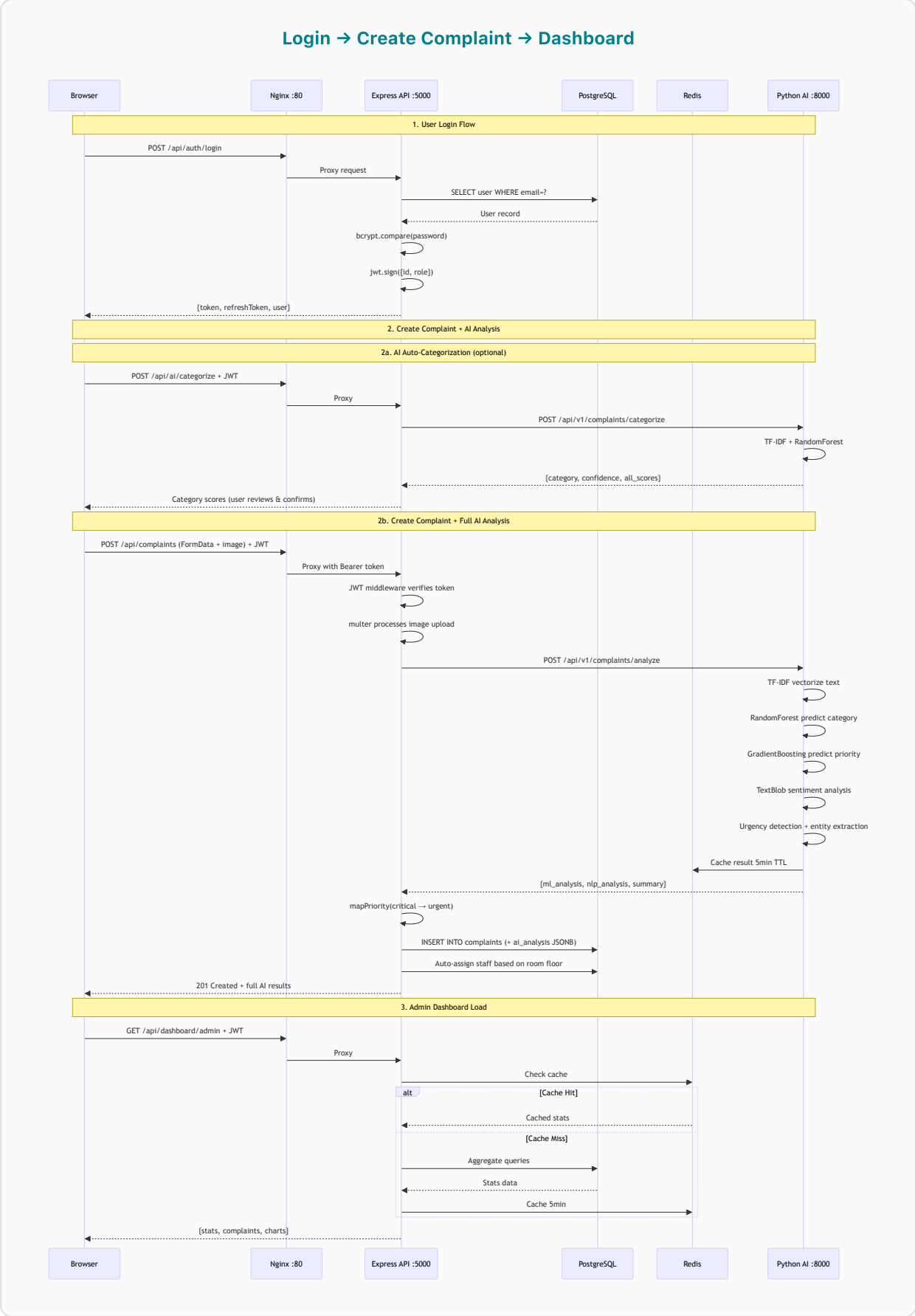


Table Details

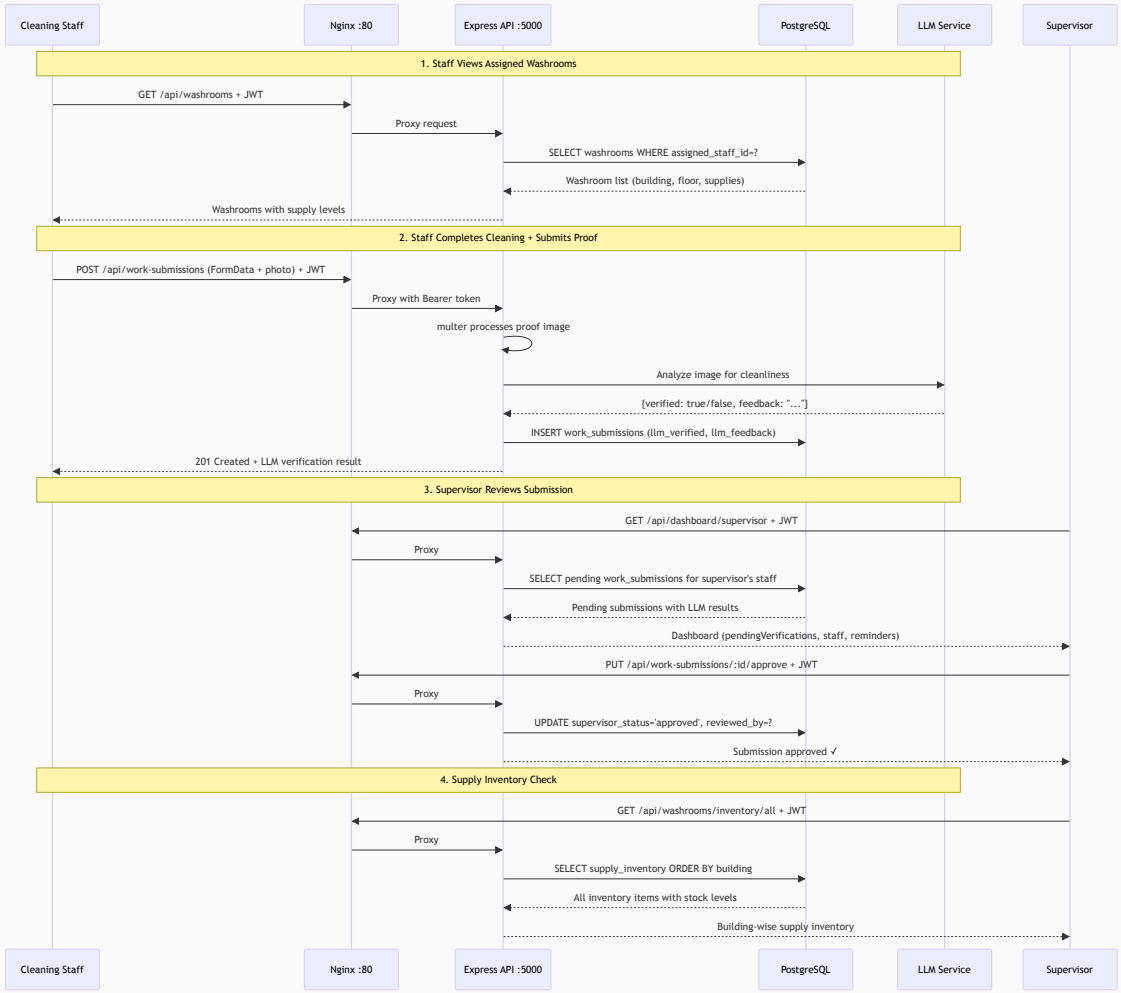
Table	PK Type	Key Columns	Indexes	Purpose
`users`	UUID	email (unique), role (admin/supervisor/staff/resident), password (bcrypt), shift, supervisor_id (FK → users), is_active	email, role, is_active	All system users — admin, supervisors, cleaning staff, students/faculty
`floors`	INT	name, building, assigned_staff_id (FK → users)	assigned_staff_id	Building floors with staff assignment
`rooms`	INT	room_number (unique), floor_id, status, resident_id, assigned_staff_id	floor_id, status, resident_id	Individual rooms with cleaning status
`complaints`	INT	category, description, building, status, priority, ai_analysis (JSONB)	status, priority, category, resident_id, created_at	Resident complaints with AI analysis results
`cleaning_logs`	INT	room_id, staff_id, status, duration_minutes	room_id, staff_id, created_at	Cleaning activity history
`notifications`	INT	user_id, title, message, type, read	user_id, read, created_at	In-app notifications
`ai_insights`	INT	type, data (JSONB), confidence	type, created_at	AI-generated insights and trends

`refresh_tokens`	INT	user_id, token (unique), expires_at, revoked	user_id, token, expires_at	JWT refresh token storage
`audit_logs`	INT	user_id, action, entity_type, details (JSONB), ip_address	user_id, action, created_at	Security audit trail
`washrooms`	INT	building, floor_number, direction (North/South/East/West), status, soap_level (0-100), tissue_level (0-100), sanitizer_level (0-100), assigned_staff_id FK	building, floor_number, status	Building/floor/direction-wise washroom tracking with supply levels
`work_submissions`	INT	staff_id FK, washroom_id FK, description, image_url, llm_verified, llm_feedback, supervisor_status (pending/approved/rejected), reviewed_by FK	staff_id, washroom_id, supervisor_status	Cleaning proof submissions with 2-way verification (LLM + supervisor)
`supply_inventory`	INT	building, item_name, current_stock, min_threshold, unit, last_restocked, restocked_by FK	building, item_name	Building-wise supply inventory tracking (soap, tissue, sanitizer)
`reminders`	INT	supervisor_id FK, building, reminder_type, message, is_active, due_date	supervisor_id, due_date, is_active	Supervisor consumption-based restocking reminders
`knex_migrations`	INT	name, batch, migration_time	—	Knex migration tracking (framework-managed)

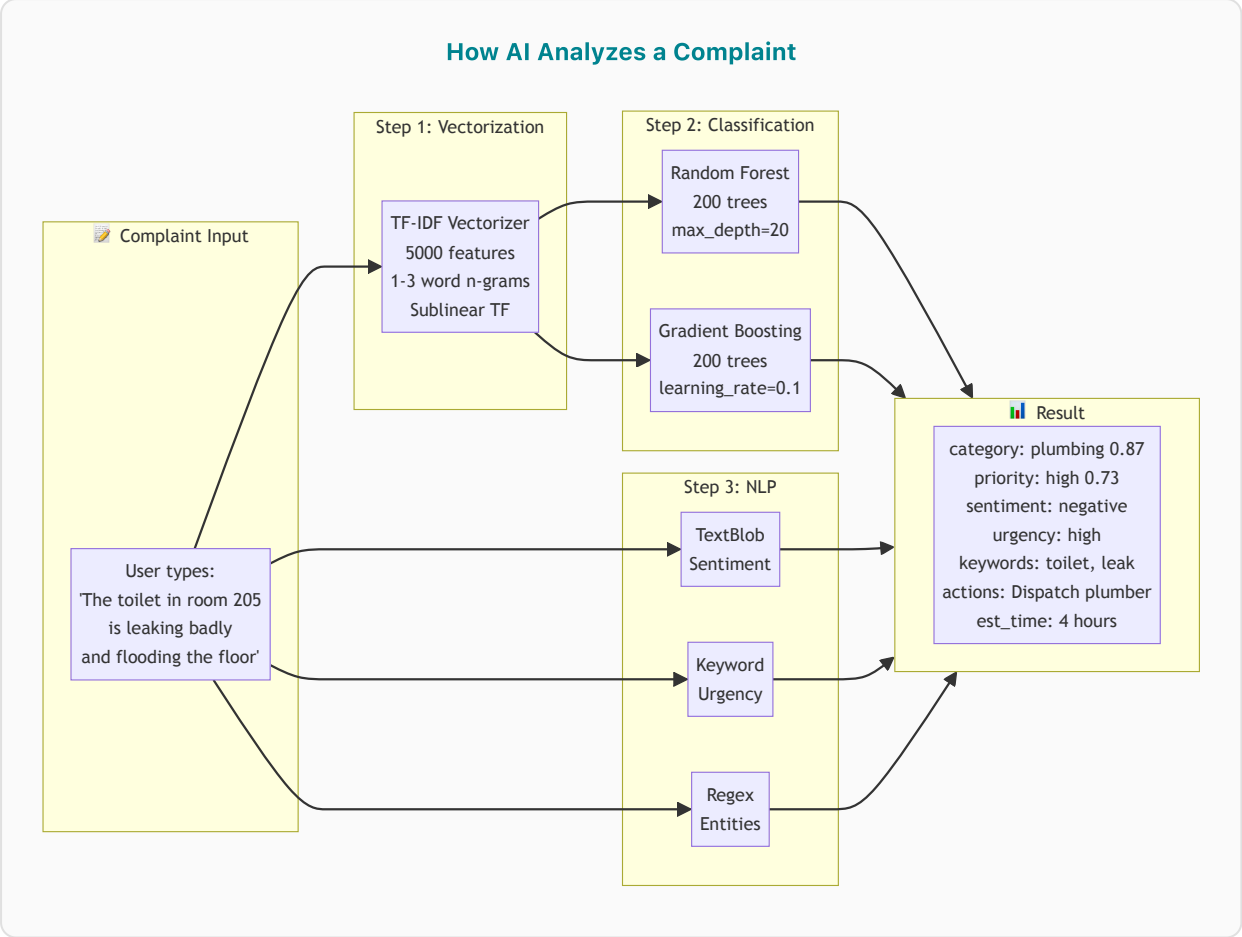
6. Request Flow Sequences



Washroom Cleaning → Work Submission → 2-Way Verification



7. AI/ML Analysis Pipeline



12 Complaint Categories

Category	Example Keywords	Suggested Actions
plumbing	leak, pipe, toilet, tap, drain, clogged, water	Dispatch plumbing team, check water damage
electrical	wire, outlet, socket, power, switch, short circuit	Send electrician, safety inspection
cleaning	dirty, stain, dust, trash, garbage, hygiene	Schedule deep cleaning
furniture	bed, chair, table, desk, wardrobe, broken	Inspect/replace furniture
pest_control	cockroach, rat, ant, mosquito, bug, insect	Schedule pest treatment
noise	loud, noise, music, construction, disturbance	Issue warning, enforce quiet hours
security	theft, lock, break in, stranger, unsafe	Notify security, install cameras
maintenance	repair, fix, cracked, worn, paint, wall	Schedule maintenance crew
internet	wifi, internet, network, router, slow, disconnect	Reset router, check ISP
appliance	ac, fan, geyser, fridge, washing machine	Technician inspection
hvac	heating, cooling, ventilation, temperature	HVAC technician dispatch

other	general, miscellaneous	Manual review required

8. Every File Explained

8a. Root Level Files

File	Lines	Language	What It Does
`index.html`	3,884	HTML JS	**The entire web frontend** — track-based login entry (Admin / Housekeeping / Faculty-Student), admin dashboard, supervisor dashboard, washroom status grid (building/floor/direction-wise), supply inventory tracking, room/task modules, complaints list, staff management, modals, filters, and theme toggle. Includes role-aligned controls: admin complaint view is read-only with false-report escalation to supervisors, admin building-to-supervisor assignment, and supervisor complaint assignment to their own staff. **Fully integrated with backend API** — real-time complaint CRUD, FormData image upload, AI auto-categorization flow, complaint detail modal with AI analysis, and dashboard refresh logic
`docker-compose.yml`	141	YAML	Defines all 6 Docker containers (PostgreSQL, Redis, Node API, Python AI, Nginx, Certbot), their ports, volumes, health checks, environment variables, and dependency ordering
`deploy.sh`	96	Bash	One-command deployment — generates secrets (JWT, DB password, Redis password via openssl), creates self-signed SSL certs, copies frontend to frontend-dist/, builds Docker images, starts containers, runs DB migrations
`.env.example`	80	Env	Template for root-level environment variables: DB credentials, JWT secrets, Redis password, AI API key, CORS, SMTP, upload, rate limit settings
`README.md`	~100	Markdown	Project documentation

8b. Backend — Node.js API (backend/)

Core Files

File	Lines	What It Does
`server.js`	221	**Express app entry point** — configures: helmet (security headers), CORS (configurable origins), rate limiting (100 req/15min API, 20 req/15min auth), compression (gzip), morgan (HTTP logging), body parsing (10MB limit). Mounts 9 route modules (auth, complaints, rooms, staff, dashboard, ai, washrooms, work-submissions, reminders). Implements cluster mode (4 workers in production). Serves frontend static files. Health check (/health) and readiness (/ready) endpoints. Graceful shutdown on SIGTERM/SIGINT
`config/index.js`	97	**Centralized configuration** — reads ALL environment variables into structured config object with sections: app (port, host, env), jwt (secret, expiry), db (host, port, pool min/max, SSL), redis, ai (serviceUrl, apiKey), upload (maxSize, allowedTypes), rateLimit, cors, log, metrics. Validates critical vars in production
`config/knexfile.js`	~30	Knex migration config — specifies migration directory, seed directory, PostgreSQL client

`package.json`	~50	Dependencies: express, knex, pg, ioredis, jsonwebtoken, bcryptjs, helmet, hpp, cors, multer, joi, winston, morgan, compression, express-rate-limit, dotenv, axios
`Dockerfile`	~25	Multi-stage build: Stage 1 (builder) runs npm ci --production; Stage 2 (node:20-alpine) copies node_modules, app code, runs as non-root user with tini for PID 1 signal handling

Database Files

File	Lines	What It Does
`database/postgres.js`	78	**PostgreSQL connection via Knex** — creates connection pool (min:2, max:20, 30s acquire timeout), health check (SELECT 1), query logging in dev, pool event logging, graceful destroy
`database/migrations/20240301000001_initial_schema.js`	169	**Creates initial 10 tables**: users (UUID PK, email unique, bcrypt password, role enum), floors, rooms (status enum, foreign keys), complaints (JSONB ai_analysis, priority/status enums), cleaning_logs, notifications, ai_insights, refresh_tokens, audit_logs. Each table has appropriate indexes
`database/migrations/20240302000001_hygiene_monitoring.js`	132	**Adds 4 new tables + schema enhancements**: Adds 'supervisor' to user role enum. Adds shift (morning/evening/night) and supervisor_id (self-FK) to users. Creates washrooms table (building, floor, direction, supply levels 0-100, last_restocked). Creates work_submissions (staff proof with LLM verified + supervisor approval). Creates supply_inventory (building-wise stock tracking). Creates reminders (supervisor consumption-based). Adds building column to complaints. Widens room_number columns to varchar(200)
`database/seeds/001_initial_data.js`	201	**Seeds comprehensive demo data**: 1 admin (admin@hostel.com), 2 supervisors (Meera Desai — morning, Suresh Nair — evening), 4 cleaning staff (with shift assignments + supervisor mapping), 4 residents (3 students + 1 faculty). 20 washrooms across 6 buildings (H1, H2, H3, Academic Block, Library, Cafeteria — H2 has North/South Wing directional data). 4 work submissions (mix of LLM verified/pending, approved/pending). 17 supply inventory items. 4 supervisor reminders. 7 hygiene-themed complaints (including false_reporting). 5 notifications. All passwords: password123 (bcrypt hashed)
`scripts/initDb.js`	~50	Runs Knex migrations (latest) then seeds. Called during deployment and via `npm run init-db`

Middleware Files

File	Lines	What It Does
`middleware/auth.js`	64	**JWT authentication** — `authenticateToken` : extracts Bearer token from Authorization header, verifies with jsonwebtoken, attaches req.user = {id, role}. `requireRole(role)` : restricts routes to specific roles (admin, supervisor, staff, resident)

`middleware/errorHandler.js`	66	**Global error handler** — catches all errors, maps: DB duplicate key → 409 Conflict, foreign key violation → 400, validation error → 400, JWT error → 401/403. Returns consistent JSON: {error, details}
`middleware/validator.js`	102	**Request validation using Joi** — schemas for: login (email + password), register (name + email + password + role), complaint (category + description + room_number), room status update, password change. `validate(schema)` middleware function
`middleware/requestLogger.js`	38	Logs every request: method, URL, response time (ms), user ID if authenticated, status code

Route Files (API Endpoints)

File	Lines	Endpoints	What It Does
`routes/auth.js`	219	POST /login, POST /register, GET /me, PUT /me, PUT /password, POST /refresh	User authentication — bcrypt password verification, JWT token generation (access 7d + refresh 30d), profile CRUD, token refresh with rotation
`routes/complaints.js`	339	GET /, POST /, GET /:id, PUT /:id/status, PUT /:id/assign, PUT /:id/priority, DELETE /:id	Complaint CRUD — file upload (multer, 10MB limit), filtering by status/priority/category/date, pagination, triggers async AI analysis on creation, requires JWT auth. Supervisors can see their assigned staff's complaints, assign complaints, and update priority. **Priority mapping**: AI may return "critical" but DB allows only low/medium/high/urgent — mapPriority() maps critical→urgent
`routes/dashboard.js`	278	GET /admin, GET /staff, GET /supervisor, GET /resident, GET /notifications, PUT /notifications/:id/read, PUT /notifications/read-all	Dashboard data — aggregates room counts by status, complaint counts by priority/status, recent items. **Supervisor dashboard**: pending verifications count, approvedToday, staff list, pending work submissions, reminders. **Admin dashboard**: now includes washroomStats (total, needsCleaning, restockedToday), verificationStats (pending, approved, rejected). Notification management. Role-specific data
`routes/rooms.js`	162	GET /, GET /by-floor, GET /my/tasks, PUT /:id/status	Room management — list with filters, group by floor, update status + auto-create cleaning_log entry. /my/tasks accessible by both staff and supervisor roles
`routes/staff.js`	152	GET /, GET /:id, GET /:id/assignments	Staff management — list all staff with stats, get individual details, view room/floor assignments. Supervisors can view their assigned staff with role/shift/supervisor_id fields
`routes/ai.js`	196	GET /status, POST /analyze-complaint, POST /categorize, GET /optimize-tasks, GET /predictions, GET /responses/:id, GET /insights	**AI integration layer** — bridges Node.js to Python AI service via HTTP, has complete fallback system (keyword-based categorization + rule-based priority) when AI service is unavailable

`routes/washrooms.js`	170	GET /, GET /by-building, GET /:id, PUT /:id/supplies, PUT /:id/status, PUT /:id/assign, GET /inventory/all, PUT /inventory/:id	**Washroom & supply management** — list all washrooms with building/floor/direction data, group by building, update supply levels (soap/tissue/sanitizer 0-100), change cleaning status, assign staff. Inventory endpoints for building-wise supply stock tracking with min threshold alerts
`routes/work-submissions.js`	217	GET /, POST /, PUT /:id/approve, GET /pending/count	**2-way work verification** — staff upload cleaning proof photos (multer), simulated LLM verifies image for cleanliness (returns verified/feedback). Supervisors approve or reject submissions. Role-based filtering: staff see own, supervisors see their team's, admin sees all. Pending count for dashboard badges
`routes/reminders.js`	134	GET /, POST /, PUT /:id, DELETE /:id, GET /due/now	**Supervisor reminders** — CRUD for consumption-based restocking reminders. Supervisors own their reminders (enforced by middleware). Due reminders endpoint returns active reminders past due date. Reminder types: restocking, inspection, follow_up, custom

Service Files

File	Lines	What It Does
`services/aiService.js`	229	**Python AI bridge** — makes HTTP calls to FastAPI (http://ai-service:8000) with 10s timeout, implements complete fallback: keyword-based categorization (maps words to categories), rule-based priority (urgency keywords), basic sentiment (positive/negative word counts). Caches results
`services/cacheService.js`	109	**Redis wrapper** — connect(), disconnect(), get(key), set(key, value, ttl), del(key), getClient(). Uses ioredis. Gracefully degrades to no-cache if Redis unavailable
`utils/logger.js`	74	**Winston logger** — console transport (colorized in dev) + rotating file transport (5MB max × 5 files). JSON format in production. Exposes morgan stream for HTTP logging

8c. AI Service — Python (ai-service/)

Core Files

File	Lines	What It Does
`main.py`	186	**FastAPI application** — CORS + GZip middleware, API key authentication (X-API-Key header), request logging with structlog (ISO timestamps, JSON in production), Prometheus metrics (request_count, latency, ml_predictions), lifespan manager (loads ML models on startup, connects Redis on start, disconnects on shutdown). Mounts 5 routers
`config.py`	55	Pydantic-style settings: MODEL_DIR, REDIS_URL, API_KEY, ALLOWED_ORIGINS, OPENAI_API_KEY, DEBUG flag, APP_VERSION

`Dockerfile`	~37	Python 3.11-slim, installs build-essential, pip install requirements.txt, downloads NLTK data to `NLTK_DATA=/usr/local/share/nltk_data` (punkt, averaged_perceptron_tagger, wordnet, punkt_tab, averaged_perceptron_tagger_eng), TextBlob corpora (full, not lite), spaCy en_core_web_sm. Runs as non-root appuser with Gunicorn + 4 Uvicorn workers. **Fixed**: NLTK_DATA path set globally so corpora are accessible to non-root appuser
`worker.py`	58	Celery worker for async tasks (batch model retraining, scheduled insights generation)
`requirements.txt`	~25	fastapi, uvicorn[standard], gunicorn, scikit-learn, numpy, pandas, textblob, nltk, spacy, openai, redis, structlog, prometheus-client, httpx, joblib, pydantic

ML Service Files

File	Lines	What It Does
`services/ml_service.py`	483	**Core ML engine** — MLService class with full lifecycle: loads pre-trained models from disk (joblib) or bootstraps training with ~100+ labeled examples. **Model 1**: TF-IDF (5000 features, 1-3 grams) → RandomForest (200 trees, max_depth=20, balanced weights) for 12-category classification. **Model 2**: TF-IDF → GradientBoosting (200 trees, lr=0.1, depth=6) for 4-level priority prediction. Also: keyword extraction via TF-IDF scores, action suggestion map (per category), resolution time estimation, cross-validation logging
`services/nlp_service.py`	174	**NLP processing** — TextBlob sentiment (polarity -1→1, subjectivity 0→1, 5 labels), keyword urgency detection (4 levels × ~15 keywords each), regex entity extraction (rooms, items like "toilet"/"AC"/"fan", locations like "bathroom"/"corridor"), room number inference from text, language quality assessment (clarity score)
`services/openai_service.py`	166	**Optional GPT-4 integration** — generates human-readable complaint responses, smart resolution suggestions, maintenance summaries. Falls back gracefully if no OPENAI_API_KEY configured
`services/predictive_service.py`	207	**Predictive analytics** — maintenance forecasting based on historical complaint patterns, peak time analysis, seasonal trends, room-level risk scoring, complaint volume projection
`services/cache_service.py`	105	**Redis cache for Python** — async connect/disconnect, get/set with JSON serialization, TTL support, caches ML predictions to avoid recomputation

AI Router Files

File	Lines	Endpoints	What It Does
`routers/complaints.py`	132	POST /analyze, POST /categorize, POST /generate-response	Full ML analysis pipeline, quick-categorize, AI-generated human response text
`routers/tasks.py`	130	POST /optimize, GET /workload	Cleaning task route optimization, staff workload balancing algorithms
`routers/predictions.py`	128	GET /maintenance, GET /complaints-forecast	Predictive maintenance schedules, complaint volume forecasting

`routers/insights.py`	143	GET /dashboard, GET /trends	Dashboard AI insights with recommendations, trend analysis
`routers/health.py`	36	GET /health	Service health (model status, cache status, features list, version)

8d. Mobile App — React Native (HostelCareApp/)

File	Lines	What It Does
`App.js`	293	**WebView app shell** — loads the web app URL with versioned cache-busting, host fallback logic, external-link bridge, download handling, and retry UI for network failures
`mobile-src/App.js`	293	Mirrored React Native app entry for alternate mobile workspace, kept in sync with the same WebView runtime behavior
`src/services/api.js`	129	**API client module** — fetch-based helper with token-aware headers, auth helpers, and endpoint wrappers used by legacy native screen flows
`src/screens/LoginScreen.js`	~150	Login form with email/password, role selection buttons, demo login shortcuts
`src/screens/AdminDashboard.js`	~120	Admin overview: stats cards (rooms/complaints/staff counts), recent complaints list, quick action buttons
`src/screens/StaffDashboard.js`	~100	Staff view: assigned tasks list, pending complaints, room status updates
`src/screens/ResidentDashboard.js`	~120	Resident view: own complaints history, room info, new complaint button
`src/screens/ComplaintsScreen.js`	~100	Complaint list with filter/search, status badges, assign/update actions
`src/screens/NewComplaintScreen.js`	~100	Complaint form: category picker, text description, image capture/upload
`src/screens/TasksScreen.js`	~100	Staff cleaning tasks and maintenance issue tracking
`src/screens/RoomManagementScreen.js`	~80	Room grid organized by floor, color-coded status indicators
`src/screens/ProfileScreen.js`	~80	User profile display, settings, logout functionality
`src/theme/colors.js`	~30	Color palette constants matching web theme

8e. Infrastructure Files (nginx/)

File	Lines	What It Does
`nginx/nginx.conf`	53	**Main Nginx config** — worker_processes auto, worker_connections 1024, gzip on (min 1000 bytes), rate limit zones: "api" (30 req/s per IP), "auth" (5 req/min per IP), upstream api_backend → api:5000

`nginx/conf.d/default.conf`	109	**Site configuration** — HTTP server (port 80): serves frontend + proxies /api/* to Express, SPA fallback (try_files → index.html). HTTPS server (port 443): TLS 1.2/1.3, strong ciphers, HSTS (2 years), SSL cert/key from /etc/letsencrypt, rate-limited /api/auth/ (5 req/min). Static asset caching (30 days, immutable). Certbot challenge location

9. Complete API Endpoints (50+)

Method	Endpoint	Auth	Role	Purpose
POST	/api/auth/login	No	Any	Login, returns JWT access + refresh tokens
POST	/api/auth/register	No	Any	Create new user account
GET	/api/auth/me	Yes	Any	Get current user profile
PUT	/api/auth/me	Yes	Any	Update profile (name, phone, avatar)
PUT	/api/auth/password	Yes	Any	Change password (requires old password)
POST	/api/auth/refresh	No	Any	Refresh JWT token using refresh token
Complaints				
GET	/api/complaints	Yes	Any	List complaints (filter, paginate, search)
POST	/api/complaints	Yes	Any	Create complaint + trigger AI analysis
GET	/api/complaints/:id	Yes	Any	Get single complaint with AI analysis
PUT	/api/complaints/:id/status	Yes	Admin/Staff/Supervisor	Update complaint status + assign staff
PUT	/api/complaints/:id/assign	Yes	Admin/Supervisor	Assign complaint to specific staff member
PUT	/api/complaints/:id/priority	Yes	Admin/Supervisor	Update complaint priority level
DELETE	/api/complaints/:id	Yes	Admin	Delete complaint
Rooms				
GET	/api/rooms	Yes	Any	List rooms with filters (status, floor)
GET	/api/rooms/by-floor	Yes	Any	Rooms grouped by floor
PUT	/api/rooms/:id/status	Yes	Staff/Supervisor/Admin	Update room status + create cleaning log
GET	/api/rooms/my/tasks	Yes	Staff/Supervisor	Get assigned room tasks for current user
Staff				
GET	/api/staff	Yes	Admin/Supervisor	List all staff (supervisors see their assigned staff)

GET	/api/staff/:id	Yes	Admin/Supervisor	Staff details with role, shift, supervisor_id
GET	/api/staff/:id/assignments	Yes	Admin/Supervisor	Staff room/floor assignments
Dashboard				
GET	/api/dashboard/admin	Yes	Admin	Admin stats (rooms, complaints, staff, washroomStats, verificationStats)
GET	/api/dashboard/staff	Yes	Staff/Supervisor	Staff tasks and assigned items
GET	/api/dashboard/supervisor	Yes	Supervisor	Pending verifications, approved today, staff list, pending submissions, reminders
GET	/api/dashboard/resident	Yes	Resident	Resident's complaints and room info
GET	/api/dashboard/notifications	Yes	Any	User's notifications
PUT	/api/dashboard/notifications/:id/read	Yes	Any	Mark notification as read
PUT	/api/dashboard/notifications/read-all	Yes	Any	Mark all notifications read
AI Services				
GET	/api/ai/status	Yes	Any	AI service health + available features
POST	/api/ai/analyze-complaint	Yes	Any	Full ML + NLP analysis of complaint text
POST	/api/ai/categorize	Yes	Any	Quick category prediction only
GET	/api/ai/optimize-tasks	Yes	Admin/Staff	Task optimization recommendations
GET	/api/ai/predictions	Yes	Admin	Predictive maintenance forecasts
GET	/api/ai/responses/:id	Yes	Any	AI-generated response suggestions
GET	/api/ai/insights	Yes	Admin	Dashboard AI insights + recommendations
Washrooms				
GET	/api/washrooms	Yes	Any	List all washrooms with supply levels
GET	/api/washrooms/by-building	Yes	Any	Washrooms grouped by building
GET	/api/washrooms/:id	Yes	Any	Single washroom details
PUT	/api/washrooms/:id/supplies	Yes	Staff/Supervisor/Admin	Update supply levels (soap, tissue, sanitizer 0-100)
PUT	/api/washrooms/:id/status	Yes	Staff/Supervisor/Admin	Update washroom cleaning status
PUT	/api/washrooms/:id/assign	Yes	Supervisor/Admin	Assign staff to washroom

GET	/api/washrooms/inventory/all	Yes	Supervisor/Admin	Building-wise supply inventory
PUT	/api/washrooms/inventory/:id	Yes	Supervisor/Admin	Update inventory stock level
Work Submissions (2-Way Verification)				
GET	/api/work-submissions	Yes	Any	List submissions (role-filtered: staff=own, supervisor=team, admin=all)
POST	/api/work-submissions	Yes	Staff	Submit cleaning proof (photo + description, LLM auto-verifies)
PUT	/api/work-submissions/:id/approve	Yes	Supervisor/Admin	Approve or reject work submission
GET	/api/work-submissions/pending/count	Yes	Supervisor/Admin	Count of pending submissions for badge display
Reminders				
GET	/api/reminders	Yes	Supervisor/Admin	List reminders (supervisors see own only)
POST	/api/reminders	Yes	Supervisor/Admin	Create consumption-based restocking reminder
PUT	/api/reminders/:id	Yes	Supervisor/Admin	Update reminder (message, due_date, is_active)
DELETE	/api/reminders/:id	Yes	Supervisor/Admin	Delete a reminder
GET	/api/reminders/due/now	Yes	Supervisor/Admin	Reminders past due date that are still active

10. User Roles & Permissions

Role	Login	Can Do	Dashboard Shows
Admin	admin@hostel.com password123	Manage rooms, staff, washrooms, and inventory. Assign buildings to supervisors. View complaints and complaint details with AI analysis. For false-report complaints, send message/escalation to supervisor. **Admin does not mark complaints in-progress/resolved in current UI flow**	Total stats (rooms/complaints/staff/washrooms/verifications), complaint list with AI tags, complaint detail popup, building-to-supervisor assignment controls, washroom status overview, supply inventory, AI recommendations

Supervisor	meera@hostel.com password123	Manage assigned cleaning staff and buildings. Review & approve/reject work submissions (2-way verification). View assigned-building room/staff mapping. Assign complaints in assigned buildings to their own staff. Create restocking reminders and update inventory context	Pending verifications count, approved today count, assigned staff list, assigned-building mapping cards, complaints in assigned buildings with assignment actions, active reminders, inventory visibility by building
Cleaning Staff	rajesh@hostel.com password123	Update room/washroom status (clean/maintain). Handle assigned complaints. View assigned floors/rooms/washrooms. Log cleaning activities. **Upload cleaning proof photos** — LLM auto-verifies for cleanliness. Submit work for supervisor approval	Assigned tasks, pending complaints, cleaning schedule, room/washroom status of assigned areas, own work submission history with verification status
Student/Faculty	student1@hostel.com password123	File new complaints (with image upload, building/direction selection). **AI auto-categorization**: can leave category blank, AI detects with scores, user reviews and confirms. View own complaint status with AI analysis. Update profile. View notifications. Report false cleaning/maintenance claims	Own complaints history with AI analysis, complaint status tracking, room info, uploaded images, building washroom status

11. ML Models — Detailed

Model 1: Complaint Classifier

Property	Value
Algorithm	Random Forest (ensemble of decision trees)
Number of Trees	200 (n_estimators=200)
Max Depth	20
Class Weighting	Balanced (handles imbalanced categories)
Input	Complaint text → TF-IDF vector (5000 features, 1-3 word n-grams, sublinear TF)
Output	Category (1 of 12) + confidence score (0-1) + all category probabilities
Training Data	~100+ labeled examples per category (bootstrap data)
Accuracy	~85% (3-fold cross-validation)
Saved As	/app/models/complaint_classifier.joblib

Model 2: Priority Predictor

Property	Value
Algorithm	Gradient Boosting (sequential ensemble)
Number of Trees	200 (n_estimators=200)
Max Depth	6
Learning Rate	0.1
Input	Same TF-IDF vector as classifier
Output	Priority (low/medium/high/critical) + confidence + all level probabilities
Saved As	/app/models/priority_predictor.joblib

NLP Pipeline Components

Component	Library	Input	Output
Sentiment Analysis	TextBlob	Raw text	Polarity (-1.0 to 1.0), Subjectivity (0 to 1.0), Label (very_negative → very_positive)
Urgency Detection	Custom keywords	Lowercased text	Level (critical/high/medium/low), matched keywords, score (0-1)
Entity Extraction	Regex patterns	Raw text	Locations (room numbers, floors), Items (toilet, fan, AC), room_info

Language Quality	TextBlob	Word list	is_descriptive, has_specifics, clarity_score (0-1)
Text Summary	TextBlob	Sentences	First sentence truncated to max_length

12. Security Features

Feature	Implementation	Details
Password Hashing	bcryptjs (10 salt rounds)	Passwords never stored in plaintext
Authentication	JWT tokens	Access token (7 day expiry) + Refresh token (30 day expiry)
HTTPS/SSL	Nginx TLS 1.2/1.3 + Certbot	Self-signed for dev, Let's Encrypt for production
Rate Limiting	Nginx + Express	100 req/15min (API), 20 req/15min (auth), 30 req/s (Nginx)
Security Headers	Helmet.js	CSP, HSTS (2 years), X-Frame-Options, X-Content-Type-Options, X-XSS-Protection
Input Validation	Joi schemas	All request bodies validated before processing
SQL Injection Prevention	Knex parameterized queries	No raw SQL string concatenation
CORS	cors middleware	Configurable allowed origins via CORS_ORIGINS env var
API Key Auth	X-API-Key header	AI service requires internal API key
Non-root Containers	Docker USER directive	Both API and AI containers run as appuser
Parameter Pollution	hpp middleware	Prevents HTTP parameter pollution attacks
Request Size Limit	Express body parser	10MB max for JSON/URL-encoded bodies
File Upload Security	Multer	10MB limit, allowed types: jpeg, png, webp only
Audit Logging	audit_logs table	Tracks user actions with IP address

13. Deployment Guide

Prerequisites

- Docker & Docker Compose V2
- macOS, Linux, or Windows with WSL2

One-Command Deploy

```
cd "/Users/maruteymani/Documents/ILGC Tracker"
chmod +x deploy.sh
./deploy.sh
```

What deploy.sh Does (step by step)

1. Checks Docker & Docker Compose installed
2. Creates `.env` from template with auto-generated secrets (JWT, DB password, Redis password)
3. Generates self-signed SSL certificates
4. Copies `index.html` → `frontend-dist/`
5. Builds all Docker images (`docker compose build --no-cache`)
6. Starts 6 containers (`docker compose up -d`)
7. Waits for PostgreSQL to be ready
8. Runs database migrations and seeds

Access After Deploy

Service	URL
Web App	http://localhost
API (direct)	http://localhost:5000
AI Service (direct)	http://localhost:8000
PostgreSQL	localhost:5432
Redis	localhost:6379

Useful Commands

```
docker compose ps            # Check container status
docker compose logs -f       # Live logs (all services)
docker compose logs api      # Logs for specific service
docker compose down          # Stop everything
docker compose up -d         # Start everything
docker compose restart nginx # Restart single service
docker compose exec -T api node scripts/initDb.js  # Re-run migrations
```

14. File Count Summary

22
Backend (Node.js)

14
AI Service (Python)

1
Frontend (HTML/JS)

13
Mobile (React Native)

Category	Files	Total Lines	Languages
Backend (Node.js)	22 files	~3,600	JavaScript (ES6+)
AI Service (Python)	14 files	~2,100	Python 3.11
Frontend	1 file	3,884	HTML5 + CSS3 + JavaScript
Mobile App	13 files	~8,080	JavaScript (React Native)
Infrastructure	5 files	~400	YAML, Bash, Nginx conf

Config/Env	4 files	~250	Env, JSON
TOTAL	**59 files**	**~11,500+**	**5 languages**

Demo Credentials:
Admin: admin@hostel.com / password123
Supervisor: meera@hostel.com / password123
Cleaning Staff: rajesh@hostel.com / password123
Student: student1@hostel.com / password123

15. Changelog

v3.1.0 — 14 March 2026 (Role Workflow Alignment + App Sync)

> **Summary:** Updated role workflow alignment across website and app shell parity. Admin complaint actions were made read-only for status updates, with false-report escalation to supervisor. Supervisor workflow expanded with assigned-building mapping and complaint-to-staff assignment. React Native app shell and alternate mobile workspace were synchronized to load latest website build with versioned cache busting. Installable debug/release APK artifacts generated.

Frontend — index.html (3,150 → 3,884 lines, +734 lines)

Change	Details
Track-Based Login Entry	Login entry now uses 3 tracks: Admin, Housekeeping (Supervisor + Cleaning Staff), and Faculty-Student. Role-track matching is enforced in UI flow
Admin Complaint Controls Tightened	Removed admin "In Progress" and "Resolve" actions from complaint cards and detail modal. Added "Write to Supervisor" action only for false-report complaints
Supervisor Operations Expanded	Added assigned-building room/staff mapping section and complaints-in-my-buildings section with assign-to-staff action for supervisor-owned teams
Assignment UX Update	Admin assignment modal changed from staff-floor assignment to building-to-supervisor assignment

Mobile — React Native App Shell Sync

Change	Details
HostelCareApp/App.js	WebView shell cache version bumped and URL parity maintained with deployed frontend copy
mobile-src/App.js	Synced to same WebView shell behavior and versioning strategy
APK Packaging	Generated installable Android artifacts: release and debug APK for device installation/testing

Current vs Historical Notes

> **Current behavior (v3.1.0):** Admin complaint status actions are read-only, with false-report escalation via "Write to Supervisor". Supervisors handle assignment inside their mapped buildings. Login entry uses 3 tracks (Admin, Housekeeping, Faculty-Student) that route to role-specific dashboards.
>
> **Historical behavior (v3.0.0 section below):** The v3.0.0 notes capture rollout-era capabilities and should be read as release history, not as the latest permission model.

v3.0.0 — 13 March 2026 (Campus Hygiene Monitoring Overhaul)

> **Summary:** Major system overhaul aligned with the campus hygiene monitoring problem statement. Added supervisor role, building/floor/direction-wise washroom tracking, supply inventory management, 2-way work verification (LLM + supervisor approval), consumption-based reminders, false reporting detection. Frontend redesigned for 4 user roles. Backend expanded with 3 new route modules, 4 new database tables, and 20+ new API endpoints.

Frontend — index.html (2,343 → 3,150 lines, +807 lines)

Change	Details
4 User Roles	Login page updated from 3 to 4 roles: Admin, Supervisor, Cleaning Staff, Student/Faculty. Supervisor demo login: meera@hostel.com. Each role has distinct dashboard and permissions
Washroom Data — Building/Floor/Direction	New washroom management section showing all washrooms organized by building (H1, H2, H3, Academic Block, Library, Cafeteria), floor number, and wing direction (North/South/East/West). Supply level indicators for soap, tissue, sanitizer (0-100%)
Supply Inventory Dashboard	Building-wise supply tracking table showing current stock, minimum thresholds, last restocked dates. Low-stock items highlighted for immediate attention
2-Way Work Verification UI	Staff can upload cleaning proof photos. LLM verification status displayed (verified/unverified with feedback). Supervisor approval queue with approve/reject buttons. Status badges: pending (yellow), approved (green), rejected (red)
Supervisor Dashboard	Dedicated supervisor view showing: pending verification count, today's approvals, assigned staff list, pending work submissions queue, active reminders, supply inventory overview
Restocking Timeline	Visual timeline showing when supplies were last restocked per building, helping identify patterns of delayed replenishment
False Reporting Detection	Complaints can be flagged as false reports. System tracks false reporting patterns to maintain data integrity
Supervisor Reminders	Consumption-based reminder system. Supervisors can set reminders tied to supply levels and buildings with custom due dates

Backend — New Database Migration (20240302000001_hygiene_monitoring.js, 132 lines)

Change	Details
Supervisor Role	Added 'supervisor' to users role enum constraint. Added `shift` (morning/evening/night) and `supervisor_id` (self-referencing FK) columns to users table
washrooms Table	New table: building, floor_number, direction (N/S/E/W), status (clean/needs_cleaning/maintenance), soap_level (0-100), tissue_level (0-100), sanitizer_level (0-100), assigned_staff_id FK, last_restocked timestamp
work_submissions Table	New table: staff_id FK, washroom_id FK, description, image_url (photo proof), llm_verified (boolean), llm_feedback (text), supervisor_status (pending/approved/rejected), reviewed_by FK

supply_inventory Table	New table: building, item_name, current_stock, min_threshold, unit, last_restocked, restocked_by FK. Tracks soap, tissue, sanitizer per building
reminders Table	New table: supervisor_id FK, building, reminder_type (restocking/inspection/follow_up/custom), message, is_active, due_date
complaints Updated	Added `building` column. Widened room_number to varchar(200) for descriptive location names like "H2 - Ground Floor South Wing"

Backend — 3 New Route Modules (+521 lines)

Change	Details
routes/washrooms.js (170 lines)	8 endpoints: CRUD for washrooms, supply level updates (soap/tissue/sanitizer 0-100), status changes, staff assignment, building-wise inventory tracking
routes/work-submissions.js (217 lines)	4 endpoints: staff photo proof upload with simulated LLM verification, role-based listing (staff=own, supervisor=team, admin=all), supervisor approve/reject, pending count for badges
routes/reminders.js (134 lines)	5 endpoints: CRUD for consumption-based reminders with supervisor ownership enforcement, due-now query for overdue alerts

Backend — Updated Existing Routes

Change	Details
routes/complaints.js	Supervisor role-based filtering (sees their staff's complaints via subquery). New endpoints: PUT /:id/assign (admin/supervisor), PUT /:id/priority (admin/supervisor)
routes/dashboard.js	New GET /supervisor endpoint with pendingVerifications, approvedToday, staffCount, staff list, pending submissions, reminders. Admin dashboard now includes washroomStats and verificationStats
routes/staff.js	Supervisors can view their assigned staff. Returns role, shift, supervisor_id fields
routes/rooms.js	/my/tasks now accepts supervisor role. Updated room status endpoint for supervisor access
middleware/validator.js	Register schema now accepts 'supervisor' role
server.js	Added imports and route registrations for 3 new modules: /api/washrooms, /api/work-submissions, /api/reminders

Data — Comprehensive Seed Data Rewrite

Change	Details
11 Users	1 admin, 2 supervisors (Meera Desai — morning shift, Suresh Nair — evening shift), 4 cleaning staff (with shift assignments + supervisor mapping), 3 students + 1 faculty member
20 Washrooms	Across 6 buildings: H1, H2, H3, Academic Block, Library, Cafeteria. H2 includes North/South Wing directional data. Various supply levels and cleaning statuses
4 Work Submissions	Mix of LLM verified/unverified, supervisor approved/pending — demonstrates the 2-way verification pipeline

17 Supply Inventory Items	Soap, tissue rolls, sanitizer bottles across all 6 buildings with realistic stock levels and thresholds
4 Reminders	Restocking and inspection reminders tied to specific buildings
7 Complaints	Hygiene-themed (empty soap, clogged drains, pest sightings) including false_reporting category

v2.1.0 — 12 March 2026 (API Integration + AI Features)

> **Summary:** Frontend fully connected to backend APIs. AI features are now real and functional — auto-categorization, priority prediction, sentiment analysis, urgency detection all work end-to-end. New complaint detail popup modal. Image upload and display. Bug fixes in backend.

Frontend — index.html (1,921 → 2,343 lines, +422 lines)

Change	Details
Full API Integration	Frontend was operating in offline/demo mode (complaints stored in local JS array, no API calls). Rewritten with real `fetch()` calls to backend — login via `/api/auth/login`, complaints via `/api/complaints`, AI via `/api/ai/categorize`
AI Auto-Categorization Flow	2-step submission when "Let AI Detect" is selected: (1) Click Submit → calls `/api/ai/categorize` → shows all 12 category scores with percentages, AI-suggested category highlighted, AI priority with confidence. (2) User reviews scores, can change category via dropdown, clicks "Confirm & Submit" to actually submit. Prevents accidental miscategorization
Complaint Detail Modal	New popup overlay (not full-screen) when clicking any complaint card in admin or staff view. Shows: resident name, room number, status badge, priority badge, full description, attached image (clickable to open full-size), complete AI analysis with confidence bars (ML category + %, ML priority + %, sentiment + polarity, urgency level), suggested actions list, estimated resolution hours, summary
Image Upload & Display	Complaint form uses `FormData` for multipart upload. Images stored in shared Docker volume. Displayed in: admin complaint cards (📷 indicator), detail popup (full image), resident dashboard
AI Tags on Complaint Cards	Admin/staff complaint cards show colored AI tags: 🤖 category with confidence %, sentiment emoji, ⚡ urgency level, priority badge. Visual at-a-glance AI information
Category Made Optional	Category dropdown default: "🤖 Let AI Detect Automatically". Expanded from 6 to 12 categories matching backend ML model. Category label shows "(optional — AI will detect)"
New CSS	~120 lines added: `.ai-analysis-box`, `.ai-tag` variants (category, priority-*, sentiment), `.confidence-bar`, `.detail-modal`, `.detail-grid`, `.detail-ai-grid`, `.detail-ai-card`, `.detail-image`, `.spinner` animation, disabled button styles
Double-Submission Prevention	Global `_submitting` guard blocks duplicate API calls. Button disabled with spinner during submission. Submit button hidden after successful submission until form reset

Backend — routes/complaints.js

Change	Details

Bug Fix: AI Analysis Path	`analysis?.priority?.priority` → `analysis?.ml_analysis?.priority?.priority` (and same for category). The AI service returns nested structure under `ml_analysis`, but the backend was reading from wrong path, so AI-suggested priority/category were never being applied to the DB row
Priority Mapping	Added `mapPriority()` function. DB check constraint allows only: low, medium, high, urgent. AI model can return "critical". Mapping: critical→urgent, unknown→medium. Prevents `complaints_priority_check` constraint violations

AI Service — Dockerfile

Change	Details
Bug Fix: NLTK Corpora Path	NLTK corpora were downloaded as root during Docker build but app runs as `appuser`. Added `ENV NLTK_DATA=/usr/local/share/nltk_data` and `-d $NLTK_DATA` flag to download command. Changed from `textblob.download_corpora lite` to full download. Fixes `MissingCorpusError` that was causing AI service 500 errors on the `/analyze` endpoint

Data — AI Analysis Backfill

Change	Details
Seed Complaints Updated	All 8 seed complaints that were created before AI integration now have full AI analysis data (ml_analysis + nlp_analysis + summary) populated via backfill script that called the AI analyze endpoint and updated the DB

ILGC Tracker v3.1.0 — Complete System Documentation

Generated: 14 March 2026 • 59 source files • ~11,500+ lines of code • 5 languages